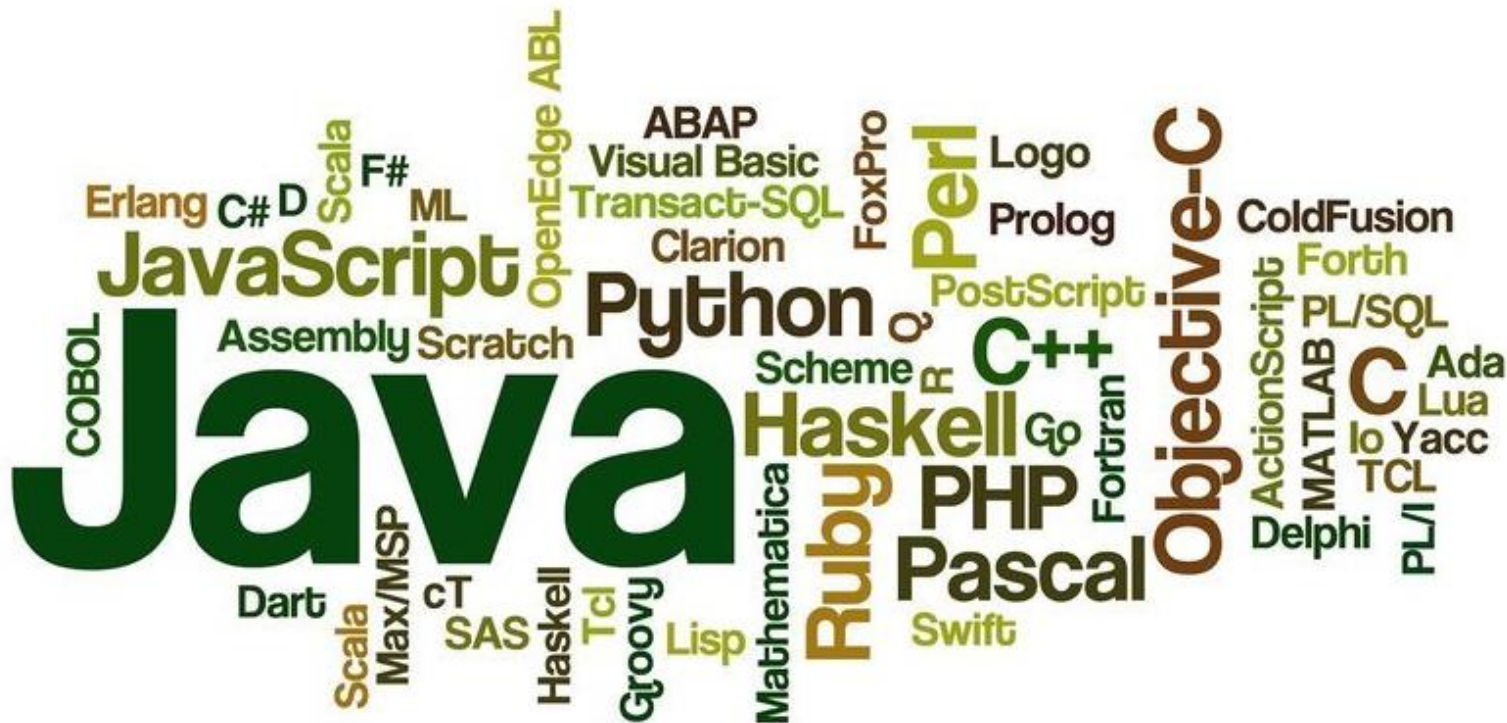


TOPIC 02

PROGRAMMING CONCEPTS

Fundamentals of Computing
(FSPK0022)



"Everybody should learn how to program a computer...
because it teaches you how to think."

Steve Jobs

Computer Languages



*Computer were programmed through switches (input)
Result through blinking light (output)*

- First programmable computers required the programmers to write explicit instructions to directly manipulate the hardware of the computer.
 - This “machine language” was very tedious to write by hand
 - Machine language is often referred to as a “low level language” since the code directly manipulates the hardware of the computer.
- By contrast, higher level languages such as C, C++, Pascal, Cobol, Fortran, ADA and Java are called “compiled languages”.
- In a compiled language
 - The programmer writes more general instructions
 - **Compiler** (a special piece of software) automatically translates these high level instructions into machine language.
 - The machine language is then executed by the computer.

Computer Languages

High-level Language

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
TEMP = V(K)
V(K) = V(K+1)
V(K+1) = TEMP
```

C/Java Compiler

Fortran Compiler

Assembly Language

```
lw St0, 0($2)
lw St1, 4($2)
sw St1, 0($2)
sw St0, 4($2)
```

MIPS Assembler

Machine Language

```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 1000 0000 1001
0101 1000 0000 1001 1100 0110 1010 1111
```

Source
Code

COMPILER

- All code is translated before execution
- An executable file is created at the time of compilation and the program is run later.
- Program executes faster than other types
- Changes to the source code requires re-compilation, recreation of the executable.

INCREMENTAL COMPILER

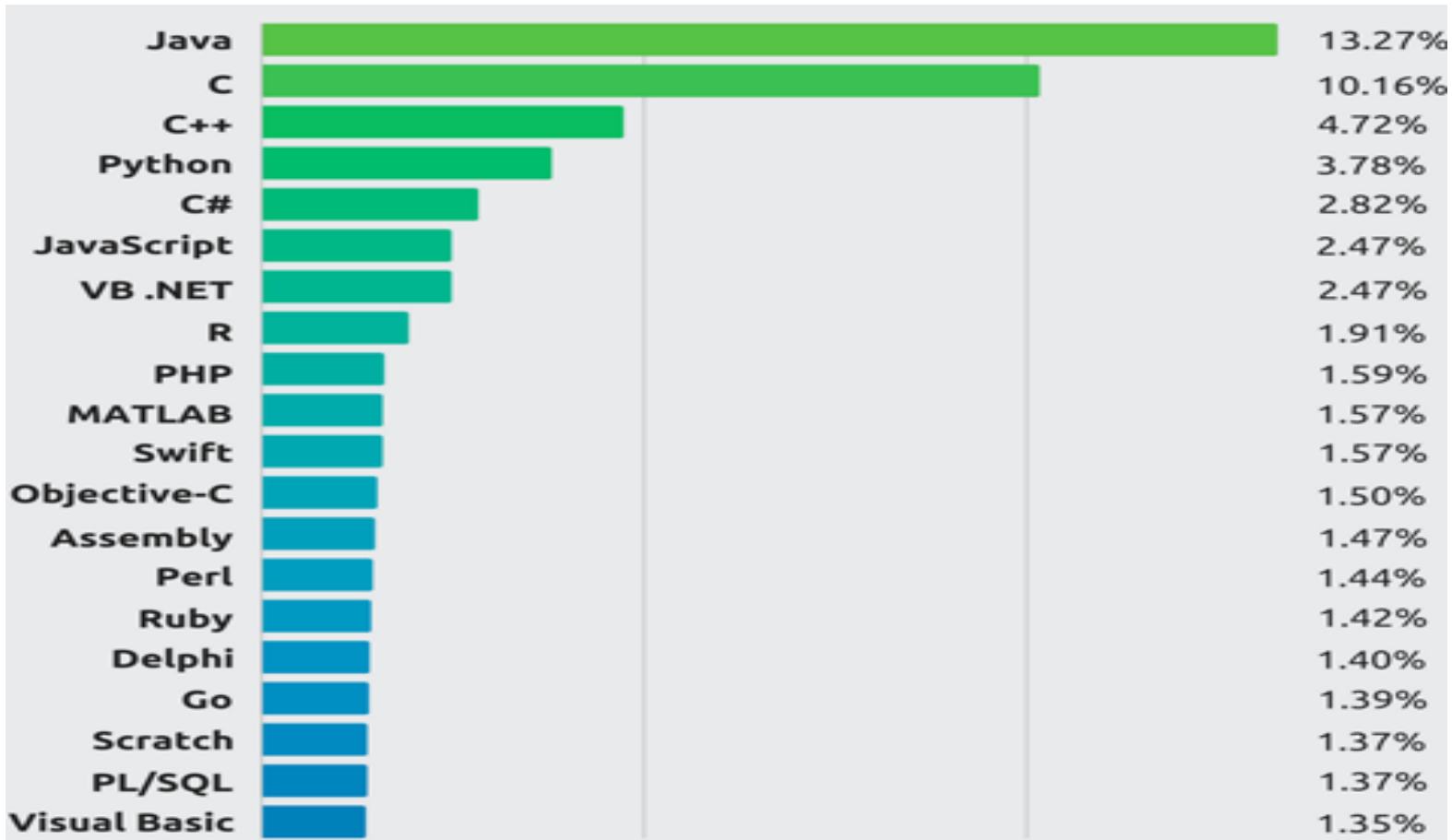
- A compromise between the other two types
- High-level code is interpreted. Sections of code used repeatedly are compiled and stored for later use by the interpreter as needed.

INTERPRETER

- When the program is executed it takes one instruction at a time and finds the equivalent machine code instruction.
- Each time the program is run, translated must occur, therefore execution is slow
- Easiest error correction

Object
Code

Top Programming Languages



Low Level Languages

Generation	Needs	Positive Features	Negative Features
First Generation: Direct Machine Code	Nothing - Machine language is directly understood by the computer. 0011000000111...	- Speedy processing - Can use Hex to reduce errors - Programmers has complete control of hardware - Still use to program ROM	- Processor dependent - Lengthy coding when HW changes - Requires veru accurate programming
Second Generation: Assembler, SAP, SPS	Assembler program - To translate memories into binary - ADD x, 100	- Mnemonic codes - Greater programming accuracy - Symbolic references rather than direct references (e.g storage location)	- Processor dependent - Less code that machine language

High Level Languages

Generation	Needs	Positive Features	Negative Features
Third Generation: COBOL, FORTRAN, VB, Pascal, Ada, C, C++, BASIC	Translator (Intepeter or Compiler)	• Processor independent • Programmer requires minimal HW knowledge • Uses 3 basic control structures	• Source code is converted to object code, therefore slower • Language developed for specific purposes therefore few standard versions
Fourth Generation: Dbase, SQL, LOGO	Translator	• As for third gen, also intervention, form filing and computer aided graphics	• Many are database and data dictionary dependent
Fifth Generation: USP, Prolog, ICOT !	Translator	• Utilises inference engines, knowledge bases and expert system structures	• Often processors dependent • Requires parallel processors • Has not evolved past the experimental stage

Sample Program for Different Programming Languages

- Different programming language have a different **syntax**
 - a **“correct”** way to write down codes (source code)
 - allows the programming language understand, what you’re trying to tell it to do
- A program’s source code must have correct syntax in order to compile correctly
 - Otherwise, it will fail to compile and produce a **"syntax error"**
 - A syntax error can be as simple as a **missing parenthesis** or a **forgotten semicolon** at the end of a statement.
- Syntax errors are also called **compile-time errors**, since they can prevent a program from compiling.
- Errors that occur in a program when the program is running are called **runtime errors**.

Language	Example program
"C"	<pre>#include <stdio.h> void main() { printf("Hello World"); }</pre>
C++	<pre>#include <iostream> int main() { cout << "Hello World"; return 0; }</pre>
Pascal	<pre>program helloworld (output); begin writeln('Hello World'); end.</pre>
Oracle PL/SQL	<pre>CREATE OR REPLACE PROCEDURE helloworld AS BEGIN DBMS_OUTPUT.PUT_LINE('Hello World'); END;</pre>
Java	<pre>class helloworld { public static void main (String args []) { System.out.println ("Hello World"); } }</pre>
Perl	<pre>#!/usr/local/bin/perl -w print "Hello World";</pre>
Basic	<pre>print "Hello World"</pre>

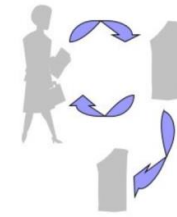
*All program display **Hello World** to the monitor*

Models of Programming

- **Object oriented programming (OOP)**

- First step, **identify the component** called **object** which form the basis of the solution.
- Determine how object interact with one another.

■ Procedural



Withdraw, deposit, transfer

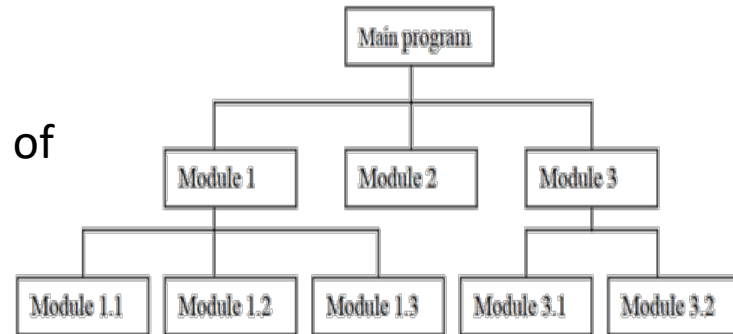
■ Object Oriented



Customer, money, account

- **Procedural programming (PP)**

- Programmer specifies the sequence of operation to be performed.
- **Dividing a problem into smaller subproblems.**
- Each subproblem is then analyzed and solution obtained to solve subproblem.
- The solution of subproblems combined to solve the overall problem.



Structure chart

Procedural vs Object Oriented Programming

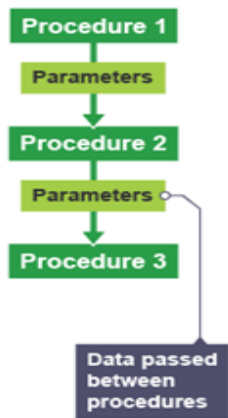
Procedural Programming (PP)	Object Oriented Programming (OOP)
Program is divided into small parts called functions .	Program is divided into parts called objects .
PP does not have any access specifier	OOP has access specifiers named public, private, protected , etc
PP does not have any proper way for hiding data, so it is less secure	OOP provides Data Hiding so provides more security
Most function uses global data for sharing that can be accessed freely from function to function in the system	Data can not move easily from function to function, it can be kept public or private so we can control the access of data
Example: C, VB, FORTRAN, Pascal	Example: C++, Java, VB.NET, C#.NET

Different Approach of Solving Problem between PP and OOP

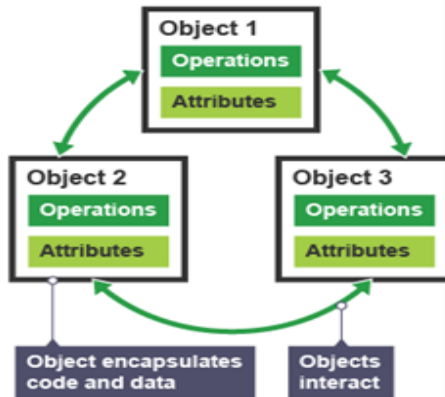
Procedural Programming (PP)

- Program starts with a problem and then **breaks** that problem down into **smaller sub-problems** or **sub-procedures**.
- These sub-procedures are continually broken down in the process called functional decomposition until the sub-procedure is simple enough to be solved.

Procedural language



Object-oriented language

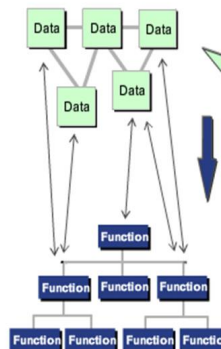


Object Oriented Programming (OOP)

- **Uses classes and objects** to create models based on the real world environment.
- Use a collection of objects which will pass messages when called upon to request a specific service or information.
- Objects are able to pass, receive messages or process information in the form of data.

Procedural:

Separation of data and functions

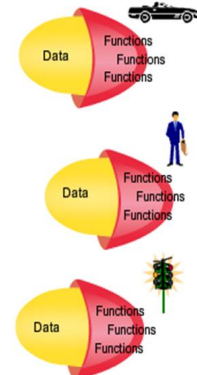


Real world



Object-oriented:

Encapsulation of data and functions



How a Program Can Solve a Problem

- **How do we write a program?**
 - A computer is not intelligent.
 - But, a computer frees people from repetitive and boring tasks.
- **Software Life Cycle Phases**
 - **Problem-Solving Phase**
 - Analyze the problem and specify what the solution must do.
 - Develop a general solution (**algorithm**) to solve the problem.
 - Algorithm: step by step procedure for solving a problem
 - Verify that your solution really solves the problem.

Mild chicken tikka masala



By Matthew Martin

10 minutes preparation time

30 minutes cooking time

Serves 4-6

Ingredients

- 500g boneless chicken
- 2 tbsp vegetable oil
- Half an onion or 2 shallots
- 2 cloves garlic
- 1 large jar tikka masala sauce
- 100ml plain yoghurt

To garnish

- Flat leaf parsley
- A lemon (cut into wedges)

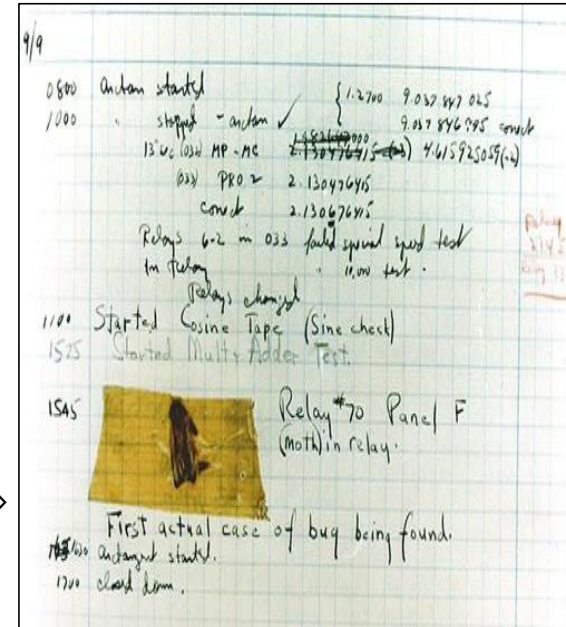
Method

1. Fry the onion and garlic gently in the oil. Add the chicken pieces and fry until golden brown and cooked all the way through.
2. Add the sauce and simmer for 5 minutes.
3. Add the yoghurt and stir through the sauce.
4. Serve the chicken with rice.

How a Program Can Solve a Problem

- Implementation Phase

- **Translating** your **algorithm into a programming language** is called **coding**.
- No single way to implement an algorithm.
- **Testing** your program means running (**executing**) your program on the computer, to see if it produces correct results.
- If it does not, then you must find out what is wrong with your program or algorithm and fix it ⇒ this is called **debugging**.



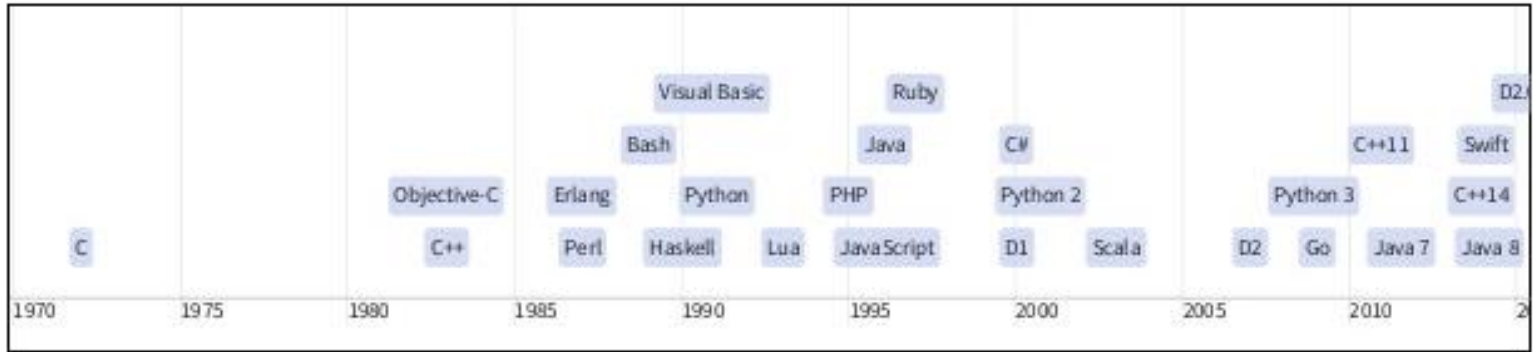
- Maintenance Phase

- Maintenance begins when your program is put into use.
- **Use** and **modify** the program to meet changing requirements or correct errors that show up in using it.

C++ Programming Language

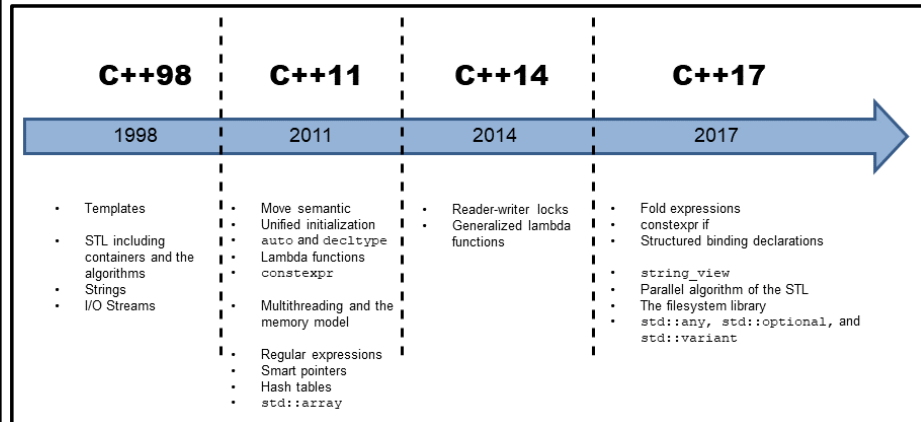
- C++ (pronounced "see-plus-plus") developed by **Bjarne Stroustrup**
- General-purpose object-oriented programming (OOP) language ⇒ **an extension of the C language** – a procedural language
- It is therefore possible to code C++ in
 - "C style"- Procedural (PP) ⇒ *use in our class, or*
 - "Object-oriented style" (OOP)
- Initially, the language was called "C with classes" ⇒ as it had all the properties of the C language with an additional concept of "classes".
- However, it was renamed C++ in 1983.
- C++ is considered to be an **intermediate-level language** ⇒ as it encapsulates both high- and low-level language features.
- Initially standardized in 1998
 - which was then amended by the C++03 and C++11 and C++14 standards.
 - The current C++17 standard supersedes these with new features and an enlarged standard library

Timeline of Programming Languages and C++



Timeline of C++

- 1979: Stroustrup starts to work on "C with Classes" in Bell Labs
- 1983: Renamed to "C++"
- 1985: The first edition of <The C++ Programming Language>
- 1989: C++ 2.0 release
- 1998: The first ISO Standard (C++98)
- 2003: C++03, bugfixes of C++98
- 2011: C++11, a major revision
- 2014: C++14, bugfixes and small improvements on C++11
- 2017: (planned) C++17, the upcoming major revision



Programming Language Structure: Overview

```
#include <iostream>
using namespace std;
```

Pre-processor directives

```
int main()
{
```

Variables declaration

Sequence
Execute line by line

```
int n, num, digit, rev = 0;
```

```
cout << "Enter a positive number: ";
cin >> num;
```

```
n = num;
```

```
do
{
    digit = num % 10;
    rev = (rev * 10) + digit;
    num = num / 10;
} while (num != 0);
```

Loop

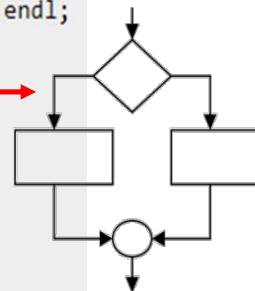
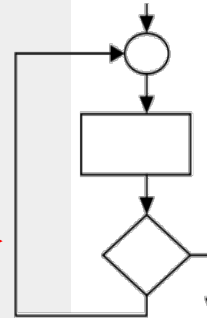
```
do statement while
(condition);
while (expression)
statement
for (initialization;
condition; increase)
statement;
```

```
cout << " The reverse of the number is: " << rev << endl;
```

```
if (n == rev)
    cout << " The number is a palindrome.";
else
    cout << " The number is not a palindrome.";
```

```
return 0;
```

```
}
```



Branch

```
if (condition)
statement
if (condition)
statement1 else
statement2
switch (expression) {
case ..
```

Programming Language

Structure: Header File

```
#include <iostream>
using namespace std;

int main()
{
    int n, num, digit, rev = 0;

    cout << "Enter a positive number: ";
    cin >> num;

    n = num;

    do
    {
        digit = num % 10;
        rev = (rev * 10) + digit;
        num = num / 10;
    } while (num != 0);

    cout << " The reverse of the number is: " << rev << endl;

    if (n == rev)
        cout << " The number is a palindrome.";
    else
        cout << " The number is not a palindrome.";

    return 0;
}
```

Header file

Header file

- Header files contain definitions of **Functions and Variables** ⇨ which is imported or used into any C++ program by using the **pre-processor #include statement**.
- Header file shows the external functionality of a program while omitting the technical implementation details.
- In C/C++, a header file gives a list of function names and how to call those functions
- The **<iostream>** header file contains code that allows a C++ program **to display output** on the screen and **read input** from the keyboard.

Programming Language Structure: Declaration

```
#include <iostream>
using namespace std;

int main()
{
    int n, num, digit, rev = 0;

    cout << "Enter a positive number: ";
    cin >> num;

    n = num;

    do
    {
        digit = num % 10;
        rev = (rev * 10) + digit;
        num = num / 10;
    } while (num != 0);

    cout << " The reverse of the number is: " << rev << endl;

    if (n == rev)
        cout << " The number is a palindrome.";
    else
        cout << " The number is not a palindrome.";

    return 0;
}
```

Variable declaration

Variables declaration

- Instructs the system to **allocate memory space** and name it
- Name is a **symbolic name for address**
- Programmer doesn't need to know the actual address
- Variables generally contain garbage if you neglect to initialize them

Programming Language

Structure: Sequence

```
#include <iostream>
using namespace std;
```

```
int main()
{
```

```
    int n, num, digit, rev = 0;
```

```
    cout << "Enter a positive number: ";
    cin >> num;
```

```
    n = num;
```

```
    do
    {
```

```
        digit = num % 10;
```

```
        rev = (rev * 10) + digit;
```

```
        num = num / 10;
```

```
    } while (num != 0);
```

```
    cout << " The reverse of the number is: " << rev << endl;
```

```
    if (n == rev)
```

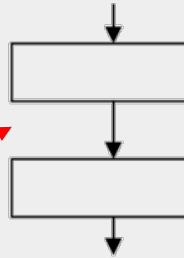
```
        cout << " The number is a palindrome.";
```

```
    else
```

```
        cout << " The number is not a palindrome.";
```

```
    return 0;
```

```
}
```



Sequence

- Unless directed otherwise, the computer executes C++ statements **one after the other** in the order in which they're written $\Rightarrow$ that is, **in sequence**.
- The program, when run, must **perform each action in order** with no possibility of skipping an action.

Programming Language

Structure: Repetition/ Loop

```
#include <iostream>
using namespace std;

int main()
{
    int n, num, digit, rev = 0;

    cout << "Enter a positive number: ";
    cin >> num;

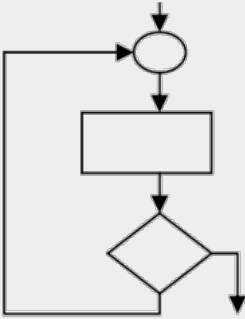
    n = num;

    do
    {
        digit = num % 10;
        rev = (rev * 10) + digit;
        num = num / 10;
    } while (num != 0);

    cout << " The reverse of the number is: " << rev << endl;

    if (n == rev)
        cout << " The number is a palindrome.";
    else
        cout << " The number is not a palindrome.";

    return 0;
}
```



Loop

do statement while (condition);
 while (expression) statement
 for (initialization; condition;
 increase) statement;

- When a **sequence of instructions** is executed in a **repeated manner**, it is called a **loop**.
- A set of instructions or structures are **repeated in a sequence** according to a **specified number of times** or **until a condition is met**
- When the set of instructions is executed again, it is called an **iteration**

Programming Language

Structure: Selection/ Branch

```
#include <iostream>
using namespace std;

int main()
{
    int n, num, digit, rev = 0;

    cout << "Enter a positive number: ";
    cin >> num;

    n = num;

    do
    {
        digit = num % 10;
        rev = (rev * 10) + digit;
        num = num / 10;
    } while (num != 0);

    cout << " The reverse of the number is: " << rev << endl;

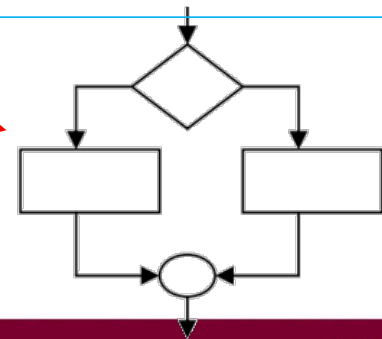
    if (n == rev)
        cout << " The number is a palindrome.";
    else
        cout << " The number is not a palindrome.";

    return 0;
}
```

Branch

```
if (condition) statement
if (condition) statement1
else statement2
switch (expression) { case ..
```

- Determines the order in which statements are executed.
- Perform different computations or actions **depending on whether a programmer-specified Boolean condition** evaluates to **True or False**



Programming Language

Structure: Function

```
#include <iostream>
using namespace std;

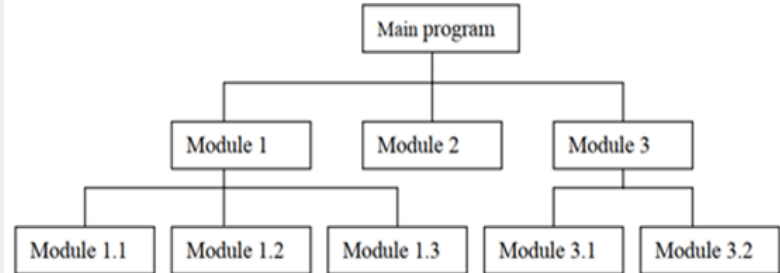
// Function prototype (declaration)
int add(int, int);

int main()
{
    int num1, num2, sum;
    cout<<"Enters two numbers to add: ";
    cin >> num1 >> num2;

    // Function call
    sum = add(num1, num2);
    cout << "Sum = " << sum;
    return 0;
}
```

```
// Function definition
int add(int a, int b)
{
    int add;
    add = a + b;

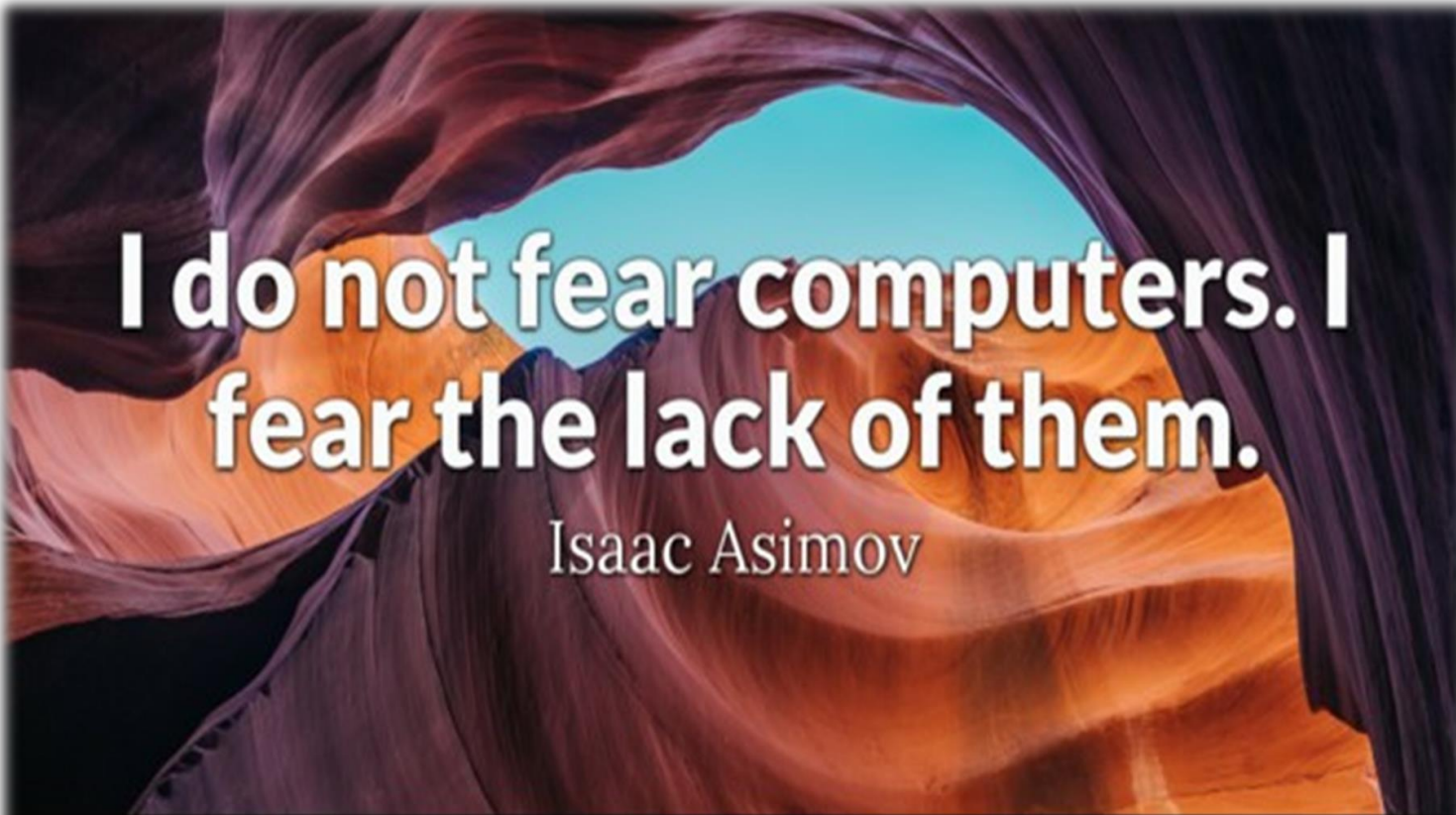
    // Return statement
    return add;
}
```



Structure chart

Function

- For a big program, need to chunk into smaller subprogram or module is called a **function**
- Function is a group of program to solve a specific problem
- **Divide and conquer** ⇒ to make problem solving more manageable



**I do not fear computers. I
fear the lack of them.**

Isaac Asimov

